

Optimizing Industrial Operations Through Predictive Maintenance Modeling

Author: Reeves Gonah

Date: 5/5/2026

Project: Phase 3 Project

Data Source: [AI4I 2020 Predictive Maintenance Dataset](#)

Introduction

A mid-sized (fictional) manufacturing firm, *Apex Precision Components Ltd.*, has recently completed its annual operational review and identified a consistent rise in maintenance costs and unplanned machine downtime across its production lines. The company specializes in producing high-tolerance mechanical components—such as shafts and gears, using CNC and automated machining systems similar to those represented in the AI4I 2020 Predictive Maintenance Dataset. These disruptions have led to delayed orders, increased repair expenses, and reduced overall efficiency. To address this, the company is exploring data-driven approaches to better understand the conditions under which machine failures occur, with the aim of proactively identifying high-risk operating states and optimizing maintenance decisions.

Business Understanding

Business Context

Unplanned machine downtime is one of the most expensive operational challenges in manufacturing. When a critical machine fails unexpectedly, production lines stop, delivery deadlines are missed, and emergency repairs can cost significantly more than planned maintenance.

Business Problem

The company currently relies on reactive or rigid scheduled maintenance. It is investigating use of a data-driven approach to identify high-risk operating conditions and predict failures early, enabling proactive intervention and reducing downtime.

Business Objectives

Develop a binary classification model that predicts machine failure risk from sensor readings such as temperature, torque, e.t.c. that will inform maintenance decisions by triggering inspections only when risk is elevated, reducing unplanned stoppages and repair costs.

Business Questions

- Which sensor conditions or combinations most strongly indicate an impending machine failure?
- Can a machine learning model distinguish between normal and failure-prone states with sufficient accuracy to be operationally trusted?
- What is the right balance between catching all failures and controlling false alarms?

Potential Beneficiaries

- Maintenance teams, plant managers, and production planners benefit from early warnings that enable proactive repairs
- Reliability engineers gain insight into failure drivers to improve operating policies
- Finance teams benefit from reduced emergency repair costs and more predictable maintenance budgets

Success Criteria

Because missed failures (false negatives) are more costly than unnecessary inspections (false positives), recall is prioritized. The model should capture a high proportion of failure cases (target $\geq 70\text{--}75\%$), while maintaining precision above $\sim 60\%$ to ensure alerts remain actionable. The F1-score will be used for overall model comparison.

Technical Approach

The relationships between sensor readings and machine failure are influenced by multiple interacting factors, such as temperature, torque, rotational speed, and tool wear. These relationships are unlikely to be captured effectively through simple rule-based thresholds or basic analysis. As such, a machine learning approach is appropriate to model these complex patterns and provide more reliable failure predictions.

Data Understanding

Data Source and Description

A data first approach was used, and the dataset selected was the AI4I 2020 Predictive Maintenance Dataset published on [kaggle](#) by Stephan Matzka, last updated in 2023. Description attached indicates that it contains 10,000 synthetic observations of a milling machine's sensor readings (air and process temperature, rotational speed, torque, tool wear) and product type, with a binary 'machine_failure' target.

The dataset is highly appropriate because it reflects realistic industrial signals, includes a naturally low failure rate, and provides sufficient data volume and feature variety to train, compare, and interpret classification models for predictive maintenance.

Data Load and Initial Exploration

```
# Importing the necessary libraries for this project
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.model_selection import GridSearchCV
from sklearn.tree import plot_tree
from sklearn.model_selection import cross_val_score

#Load the Data into a pandas dataframe
data = pd.read_csv("ai4i2020.csv")

#display the first 5 rows of the dataframe and the size
print(data.shape)
data.head()
```

(10000, 14)

	UDI	Product ID	Type	Air temperature [K]	Process temperature
[K]	\				
0	1	M14860	M	298.1	308.6
1	2	L47181	L	298.2	308.7
2	3	L47182	L	298.1	308.5
3	4	L47183	L	298.2	308.6
4	5	L47184	L	298.2	308.7

	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machine
failure	TWF	\		
0		1551	42.8	0
0	0			
1		1408	46.3	3
0	0			
2		1498	49.4	5
0	0			
3		1433	39.5	7
0	0			


```
dtypes: float64(3), int64(9), object(2)
memory usage: 1.1+ MB
```

Overview

The dataset contains 10,000 rows and 14 total columns, 6 of which are not of use (1 index), leaving 8 usable columns.

No row contains null values, however 2 columns are of string format which will need addressing.

Feature and Variable Distributions

Taking a look at some of the preliminary statistics, relationships and distributions:

```
# Column descriptive statistics
data.describe()
```

	UDI	Air temperature [K]	Process temperature [K]	\
count	10000.00000	10000.000000	10000.000000	
mean	5000.50000	300.004930	310.005560	
std	2886.89568	2.000259	1.483734	
min	1.00000	295.300000	305.700000	
25%	2500.75000	298.300000	308.800000	
50%	5000.50000	300.100000	310.100000	
75%	7500.25000	301.500000	311.100000	
max	10000.00000	304.500000	313.800000	

	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machine failure \
count	10000.000000	10000.000000	10000.000000	
mean	1538.776100	39.986910	107.951000	
std	179.284096	9.968934	63.654147	
min	1168.000000	3.800000	0.000000	
25%	1423.000000	33.200000	53.000000	
50%	1503.000000	40.100000	108.000000	
75%	1612.000000	46.800000	162.000000	
max	2886.000000	76.600000	253.000000	

	TWF	HDF	PWF	OSF
RNF				
count	10000.000000	10000.000000	10000.000000	10000.000000

mean	0.004600	0.011500	0.009500	0.009800
std	0.067671	0.106625	0.097009	0.098514
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000	0.000000
75%	0.000000	0.000000	0.000000	0.000000
max	1.000000	1.000000	1.000000	1.000000

Inspecting the class balance of the target variable

```
print(data['Machine failure'].value_counts())
print('\n')
print(data['Machine failure'].value_counts(normalize=True))
```

```
0    9661
```

```
1     339
```

```
Name: Machine failure, dtype: int64
```

```
0    0.9661
```

```
1    0.0339
```

```
Name: Machine failure, dtype: float64
```

Visually inspect the numeric features for distribution
#Plot on one figure to reduce redundancy

Select relevant numeric features

```
features = ['Air temperature [K]', 'Process temperature [K]',
            'Rotational speed [rpm]', 'Torque [Nm]', 'Tool wear [min]']
```

Create subplots

```
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
```

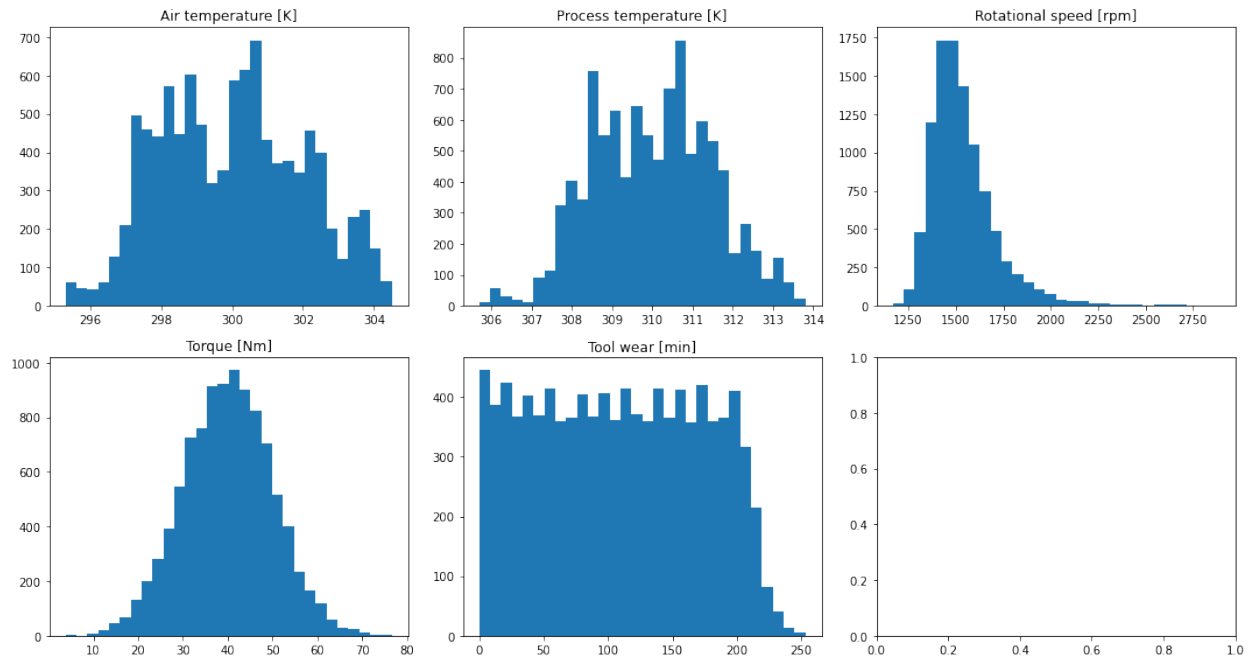
```
axes = axes.flatten()
```

Plot histograms

```
for i, col in enumerate(features):
    axes[i].hist(data[col], bins=30)
    axes[i].set_title(col)
```

```
plt.tight_layout()
```

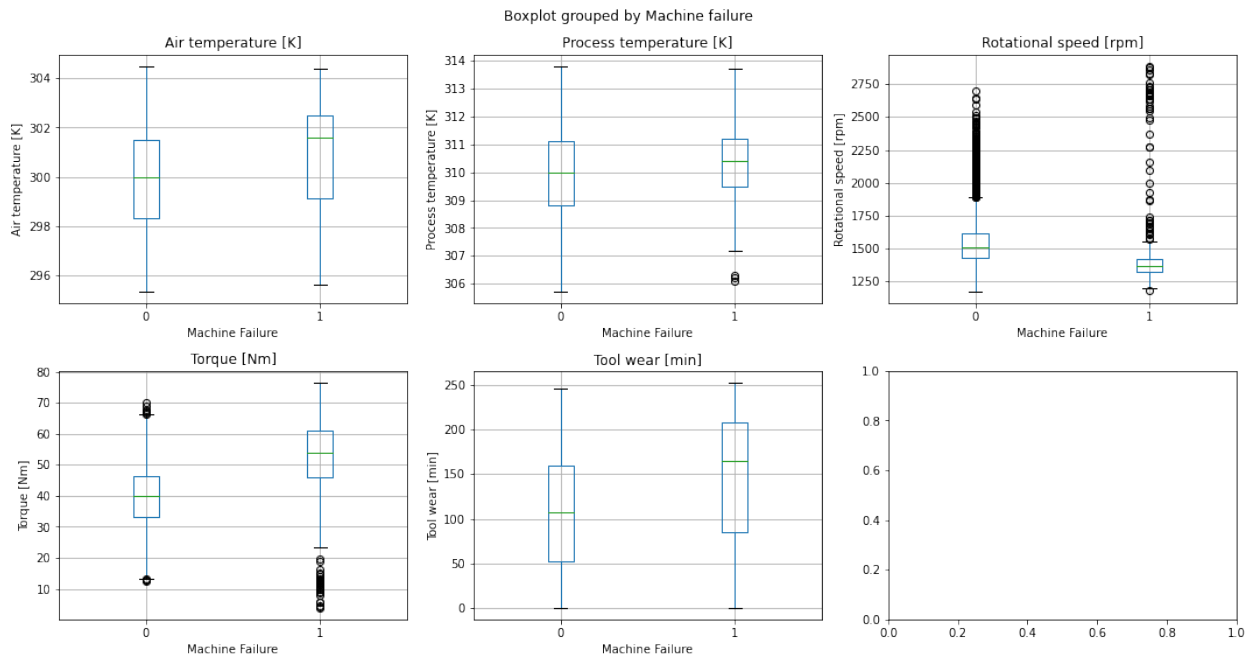
```
plt.show()
```



```
# Creating the figure and axes as previously done
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
axes = axes.flatten()

for i, col in enumerate(features):
    #using the inbuilt pandas boxplots for ease
    data.boxplot(column=col, by='Machine failure', ax=axes[i])
    axes[i].set_title(col)
    axes[i].set_xlabel('Machine Failure')
    axes[i].set_ylabel(col)

plt.tight_layout()
plt.show()
```



```
# A few features seem like they would be correlated
# (Air temp and process temp) and (torque and rotational speed)
# isolating these features
corr_features = ['Air temperature [K]', 'Process temperature [K]',
                 'Rotational speed [rpm]', 'Torque [Nm]', 'Machine failure']

# Getting the correlation matrix
corr_matrix = data[corr_features].corr()
print(corr_matrix)
```

	Air temperature [K]	Process temperature [K]
\		
Air temperature [K]	1.000000	0.876107
Process temperature [K]	0.876107	1.000000
Rotational speed [rpm]	0.022670	0.019277
Torque [Nm]	-0.013778	-0.014061
Machine failure	0.082556	0.035946

	Rotational speed [rpm]	Torque [Nm]	Machine failure
\			
Air temperature [K]	0.022670	-0.013778	0.082556
Process temperature [K]	0.019277	-0.014061	0.035946
Rotational speed [rpm]	1.000000	-0.875027	-


```

0.044188
Torque [Nm]                -0.875027    1.000000
0.191321
Machine failure            -0.044188    0.191321
1.000000

# Lastly Investigating the categorical column distribution and
# relationship to the target variable

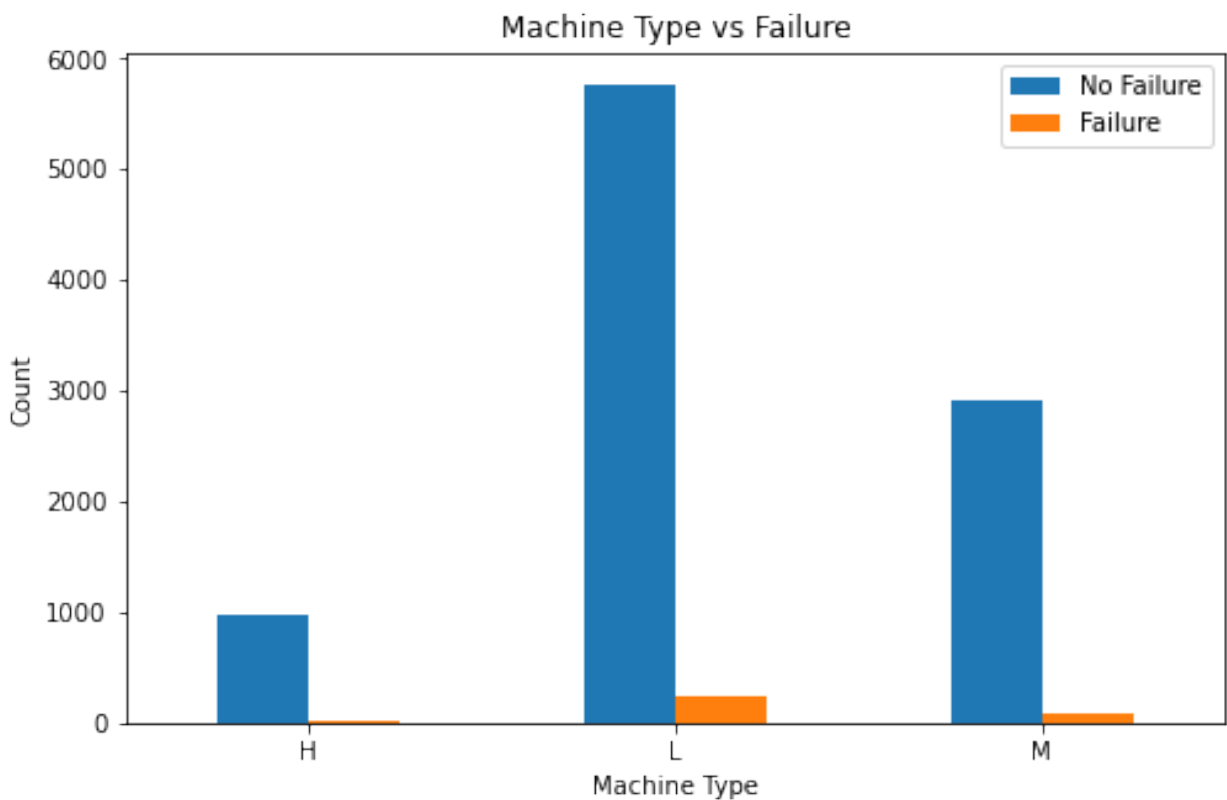
# Create grouped counts
grouped = data.groupby(['Type', 'Machine failure']).size().unstack()

# Plot
grouped.plot(kind='bar', figsize=(8, 5))

plt.title('Machine Type vs Failure')
plt.xlabel('Machine Type')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.legend(['No Failure', 'Failure'])

plt.show()

```



Observations, Patterns and Insight

1. There is class imbalance in the target variable. Failures are much fewer than non failures therefore a technique such as resampling might be necessary to avoid creating a biased model.
2. The feature distributions show that most variables are reasonably continuous, though some exhibit skewness and outliers (e.g., rotational speed and torque).
3. Machines that experienced failure tend to operate at higher torque levels and exhibit greater tool wear, indicating that mechanical stress and accumulated wear are strong indicators of failure risk.
4. The distribution of failures across machine types shows that certain machine categories (L) experience higher failure rates than others though this may be because they have the largest set of machines.
5. Torque and tool wear show stronger positive correlations with machine failure, reinforcing their importance as predictors. Additionally, air and process temperatures are highly correlated with each other, indicating potential redundancy.

Data Preparation and Preprocessing

Data Cleaning

```
# No missing values
# Drop Unnecessary columns
data.drop(columns=['UDI', 'Product ID', 'TWF', 'HDF', 'PWF', 'OSF',
                  'RNF'], axis=1, inplace=True)

# Clean and standardize column names
column_names = ['Machine_Type', 'Air_Temp', 'Process_Temp',
                'Rot_Speed', 'Torque', 'Tool_Wear', 'Machine_Failure']

#Assign new Column names
data.columns = column_names
```

Feature Engineering

```
# Create new features/ Calculated columns
# This can be done before the testing and training data is split as
# they do not rely on dataset statistics

data["Temp_Difference"] = data['Process_Temp'] - data['Air_Temp']
data["Mechanical_Power"] = data['Torque'] * data['Rot_Speed']
```

Justification for Engineered Features (based on domain knowledge):

Temperature difference: This captures the heat buildup in the machine relative to ambient conditions, which may signal excessive friction, inadequate cooling, or impending bearing failure more directly than either temperature alone.

Mechanical power: Power is a fundamental physical quantity representing the rate of work being done. Abnormally high power can indicate overloaded cutting conditions, while abnormally low power at a given speed might suggest tool disengagement or slipping. Both patterns are mechanically meaningful for predicting failures.

Splitting the training and testing data

To prevent data leakage, splitting of the dataset is done here. A standard train_test_split is being done using scikitlearns model selection library. A 70/30 split will be done to ensure a decent amount of testing data to reach a meaningful conclusion

```
# Declare the feature and target variables
X = data.drop(columns='Machine_Failure', axis=1)
y = data['Machine_Failure']

# Using a test size of 0.3 as mentioned earlier, and a random state of 73 throughout
random_state = 73

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=random_state)

# Inspect the class distribution of the train and test target variables
print("Training set: \n", y_train.value_counts(normalize=True))
print('\n')
print("Testing set: \n", y_test.value_counts(normalize=True))

Training set:
0    0.965714
1    0.034286
Name: Machine_Failure, dtype: float64

Testing set:
0    0.967
1    0.033
Name: Machine_Failure, dtype: float64
```

The class distribution remains relatively consistent across the training and testing sets.

Encoding Categorical Columns

```
# The Machine_Type column is categorical and needs to be converted
into numerical for machine learning
# The get dummies function from the pandas library
```

```

#create a function that will be used to one hot encode the test and training set seperately
def onehotencode(dataset):
    data_encoded = pd.get_dummies(dataset, columns=['Machine_Type'],
    drop_first=True, dtype=int)
    return data_encoded

# One hot encode the training set
X_train_encoded = onehotencode(X_train)

# One hot encode the test set
X_test_encoded = onehotencode(X_test)

```

Feature Scaling

The data needs to be scaled as numerical columns are not in the same units or ranges and would therefore be weighted differently, skewing feature importance and giving an incorrect model. This is particularly important for logistic regression which relies on gradient descent.

```

# The standard scaler class from scikit learn's library will be used
# Fitting is only done on the training dataset and not the testing dataset to prevent data leakage
# Initialize the scaler
scaler = StandardScaler()

#Fit and transform the training data
X_train_scaled = scaler.fit_transform(X_train_encoded)

# Transform the testing data using the model fitted on the training data
X_test_scaled = scaler.transform(X_test_encoded)

```

Class Balancing

As previously mentioned, there are very few rows containing failures as compared to those where no failure occurs, reflecting real world circumstances. To avoid having the model predict no failure most of the time and therefore be right majority of the time, the classes should be weighted. This will be done using the resampling technique

```

# The smote package from imblearn's package will be used
# instansiate
smote = SMOTE(random_state=random_state)

#fit and transform the training datasets
X_train_resampled, y_train_resampled =
smote.fit_resample(X_train_scaled, y_train)

```

```
#check the results
print((y_train_resampled).value_counts())

1    6760
0    6760
Name: Machine_Failure, dtype: int64
```

The data is now set for modelling. It would however be useful to standardize the names of the datasets

```
#Renaming the set names for ease
#Training set
X_train_clean = X_train_resampled
y_train_clean = y_train_resampled

#Testing set
X_test_clean = X_test_scaled
y_test_clean = y_test
```

Modeling

With clean data, a predictive model can now be trained. A supervised classification approach is adopted using both Logistic Regression and Decision Tree.

Logistic Regression is selected as a baseline model due to its simplicity and interpretability in modeling linear relationships between sensor variables and failure probability. The Decision Tree model to capture potential non-linear interactions that are characteristic of mechanical systems, such as stress limits and wear accumulation.

This dual-model approach allows comparative evaluation to determine which method more effectively identifies failure-prone operating conditions, while maintaining a balance between predictive performance and interpretability.

Baseline Logistic Regression Model

Because the data is already centered, the intercept is set to false, a large C to for little to no regularization, and using a MLE to get the best fit line;

```
# Using scikit-learn's Logistic Regression class
# Instantiating the class
logreg = LogisticRegression(fit_intercept=False, C=1e12,
                             solver='liblinear')

#Fitting the model
baseline_logreg = logreg.fit(X_train_clean, y_train_clean)

#Predicting on unseen data
y_hat_baseline_logreg = logreg.predict(X_test_clean)
```

Baseline Decision Tree Classifier

For this, the only hyperparameter specified was the criterion. Information gain is calculated through entropy.

```
#using Scikit-learn's Decision Tree Classifier
# Instantiating
clf = DecisionTreeClassifier(random_state=random_state,
criterion='entropy')

#Training the model
baseline_clf = clf.fit(X_train_clean, y_train_clean)

#Generating predictions
y_hat_baseline_clf = clf.predict(X_test_clean)
```

Model Evaluation

To evaluate the model, evaluation metrics such as precision, recall, accuracy and f1 score will be compared for each, and interpreted.

Baseline Logistic Regression Model

```
# using the scikit-learn metric package to calculate the evaluation
metrics
print("Accuracy:", accuracy_score(y_test_clean,
y_hat_baseline_logreg))
print("Precision:", precision_score(y_test_clean,
y_hat_baseline_logreg))
print("Recall:", recall_score(y_test_clean, y_hat_baseline_logreg))
print("F1 Score:", f1_score(y_test_clean, y_hat_baseline_logreg))
```

```
Accuracy: 0.5966666666666667
Precision: 0.07236335642802155
Recall: 0.9494949494949495
F1 Score: 0.13447782546494993
```

Observations:

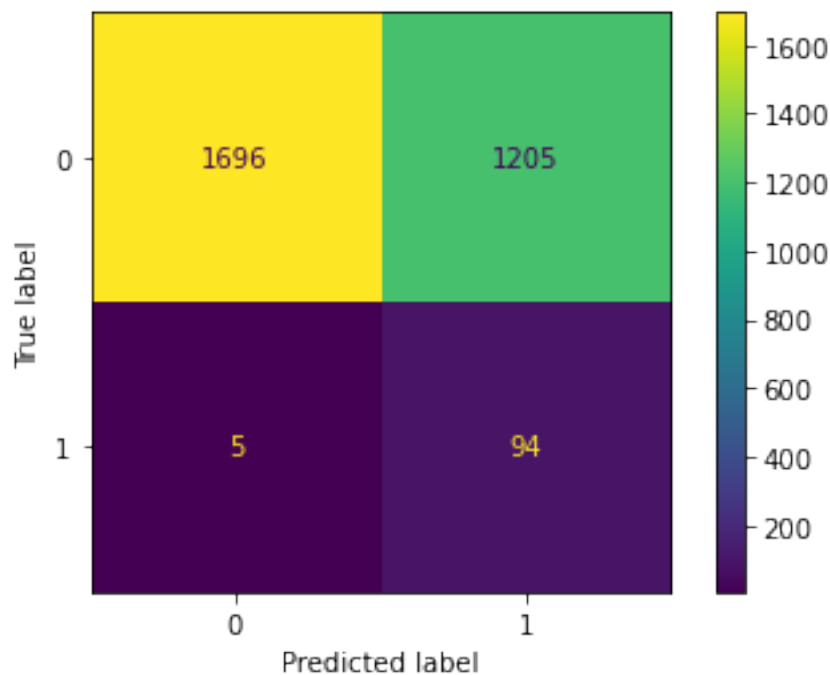
- The Logistic Regression model achieved a relatively low accuracy of about 59.7%. This could be partly influenced by class imbalance in the test set, but more importantly reflects the model's tendency to over-predict the failure class.
- The model shows very high recall (94.9%), meaning it successfully identifies most machine failures, which aligns with the business objective of minimizing missed failures.
- Precision, however, is very low (7.2%), indicating a high number of false positives which in context would lead to unnecessary maintenance alerts.

- The low F1-score (0.13) reflects this imbalance, suggesting that while the model is effective at detecting failures, it does so at the cost of poor overall precision.

Taking a look at the confusion matrix for better understanding;

```
#Getting the confusion matrix array
cm = confusion_matrix(y_test_clean, y_hat_baseline_logreg)

#plotting the display matrix using scikit learns display matrix
ConfusionMatrixDisplay(cm).plot()
plt.show()
```



- The confusion matrix shows that the model correctly identified 94 failure cases, while only missing 5 (false negatives), reinforcing its high recall.
- It however incorrectly classified 1205 non-failure instances as failures, indicating a large number of false positives.

This confirms that while the model is effective at capturing failures, it over-predicts the failure class, leading to many unnecessary maintenance alerts.

Interpretation

The logistic regression baseline prioritises recall at the expense of precision, correctly identifying nearly all true machine failures. This aligns with the core business goal of avoiding unplanned downtime, however, it generates an unacceptably high volume of false alarms. In an operational setting, this would erode trust in the model and waste resources. The poor precision and low F1-score indicate that the model is currently very aggressive in predicting the failure class.

Baseline Decision Tree Classifier

```
# using the scikit-learn metric package to calculate the evaluation metrics
print("Accuracy:", accuracy_score(y_test_clean, y_hat_baseline_clf))
print("Precision:", precision_score(y_test_clean, y_hat_baseline_clf))
print("Recall:", recall_score(y_test_clean, y_hat_baseline_clf))
print("F1 Score:", f1_score(y_test_clean, y_hat_baseline_clf))
```

```
Accuracy: 0.9673333333333334
Precision: 0.5033112582781457
Recall: 0.7676767676767676
F1 Score: 0.608
```

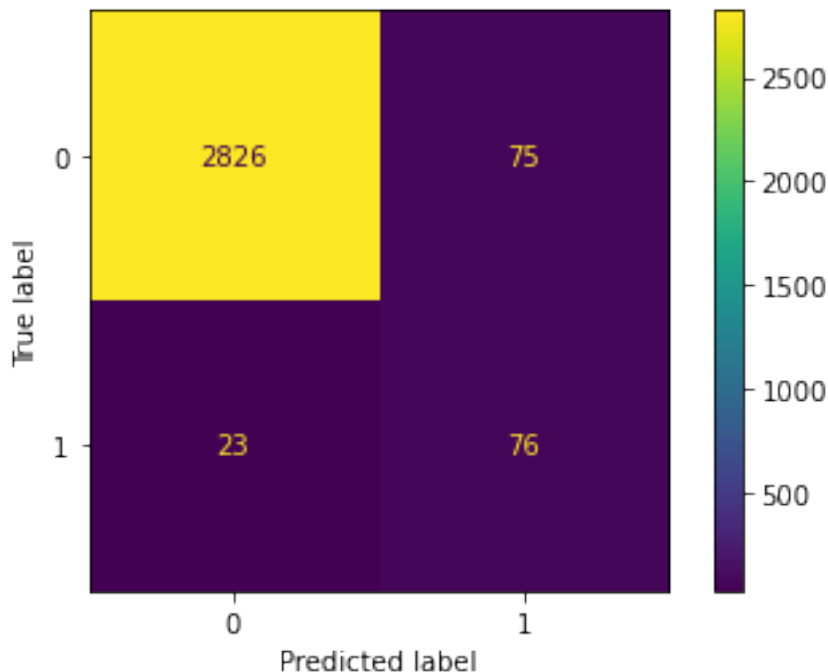
Observations:

- The Decision Tree model achieves a very high accuracy of about 96.8%, indicating that it correctly classifies the vast majority of instances, suggesting strong overall performance.
- The model shows a moderate precision (50.7%), meaning that only about half of the predicted failure cases are actually true failures.
- A 76.8% recall indicates that the model successfully detects most failure cases, reflecting a trade-off of fewer false alarms but slightly more missed failures.
- The 0.61 F1 score is reflective of the balance between precision and recall, suggesting better practical use.

Similarly, taking a look at the Confusion Matrix for better understanding

```
#Getting the confusion matrix array
cm = confusion_matrix(y_test_clean, y_hat_baseline_clf)

#plotting the display matrix
ConfusionMatrixDisplay(cm).plot()
plt.show()
```

- The model correctly caught 76 failures, missing a substantial 23 (about 30%), reinforcing relatively high recall but with a caveat.
- Most of the non failures were correctly predicted, flagging 75 as predicted failures, which is still quite high.

This provides something of a balance, but reinforces that there is tradeoff between false negative and false positive prediction.

Interpretation

The Decision Tree model achieves a more balanced performance, effectively identifying a majority of machine failures while maintaining a reasonable level of precision. This aligns well with the business objective of detecting failures without overwhelming operations with false alarms. Although quite a few failures are still missed, the model significantly limits unnecessary maintenance interventions compared to a highly aggressive approach. The higher precision and stronger F1-score indicate that the model provides more reliable and actionable predictions, indicating that it may be better suited for practical deployment with some refinement.

Model Comparison

The primary trade-off observed between the model lies in balancing false negatives (contextually missed failures) against false positives (contextually unnecessary maintenance alerts).

Although both models aim to identify failure prone conditions (recall) they differ significantly in how they manage this trade-off.

- The Logistic regression model achieves almost perfect recall at the expense of a very high number of false positives, making it a bit impractical for operational use in this state

- The decision tree classifier maintains slightly lower but still decent recall with significantly higher precision, resulting in a slightly more balanced model as reflected in the higher F1-score.

Given that achieving high recall remains the priority, contextually, a high number of false positives renders the model less useful and does not really increase efficiency. The decision tree is likely a more appropriate model for this problem, providing a better balance between reliability and actionability, making it the preferred baseline model moving forward. ____

Model Optimization

Logistic Regression Model Optimization

The recall and F1-score will be used to gauge the model's performance improvement or decline. The class weight will be set as balanced to adjust the loss function, then the regularization hyperparameter will be iterated through to find the most optimum, and lastly the threshold will also be used to control the boundary.

```
# Setting the c values to be iterated through
C_values = [0.01, 0.1, 1, 10, 100, 1000, 10000, 100000, 1000000]

# An empty list to store results
results = []
# a for loop to iterate through the logreg
for c in C_values:
    #this time with class weight set to balanced and a maximum
    iteration values set as a precaution
    improved_logreg = LogisticRegression(solver='liblinear', C=c,
    class_weight='balanced', max_iter=1000)
    improved_logreg.fit(X_train_clean, y_train_clean)
    y_hat_improved_logreg = improved_logreg.predict(X_test_clean)

    #Append results to the list
    results.append({
        "C": c,
        "accuracy": accuracy_score(y_test_clean,
y_hat_improved_logreg),
        "precision": precision_score(y_test_clean,
y_hat_improved_logreg),
        "recall": recall_score(y_test_clean, y_hat_improved_logreg),
        "F1-score": f1_score(y_test_clean, y_hat_improved_logreg)
    })
#Put the results in a pandas dataframe for easy viewing
improved_results = pd.DataFrame(results)
improved_results
```

	C	accuracy	precision	recall	F1-score
0	0.01	0.827667	0.137153	0.797980	0.234074
1	0.10	0.860667	0.164211	0.787879	0.271777
2	1.00	0.867333	0.171429	0.787879	0.281588
3	10.00	0.868667	0.172949	0.787879	0.283636

4	100.00	0.868667	0.172949	0.787879	0.283636
5	1000.00	0.868667	0.172949	0.787879	0.283636
6	10000.00	0.868667	0.172949	0.787879	0.283636
7	100000.00	0.868667	0.172949	0.787879	0.283636
8	1000000.00	0.868667	0.172949	0.787879	0.283636

There is no improvement from a value of C at 100. The precision recall has dropped slightly from the baseline, and the F1-score has improved, albeit minimally.

```
#tuning the threshold, according to scikitlearn's documentation
improves the model, default is set at 0.5
# Applying the same logic that was used in finding optimum C:

# Setting a range of threshold values
thresholds = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]

#Get the predicted probabilities for failures
y_probabilities = improved_logreg.predict_proba(X_test_clean)[: , 1]

#resetting the results array
results = []

for threshold in thresholds:
    #convert the thresholds to predicted class failure (1 or 0
    depending on the value of the threshold)
    y_pred = (y_probabilities >= threshold).astype(int)

    #Append to results
    results.append({
        "Threshold": threshold,
        "Accuracy": accuracy_score(y_test_clean, y_pred),
        "Precision": precision_score(y_test_clean, y_pred),
        "Recall": recall_score(y_test_clean, y_pred),
        "F1-Score": f1_score(y_test_clean, y_pred)
    })

threshold_results = pd.DataFrame(results)
threshold_results
```

	Threshold	Accuracy	Precision	Recall	F1-Score
0	0.1	0.569333	0.068067	0.949495	0.127027
1	0.2	0.713333	0.097354	0.929293	0.176245
2	0.3	0.787000	0.120787	0.868687	0.212084
3	0.4	0.835333	0.149201	0.848485	0.253776
4	0.5	0.868667	0.172949	0.787879	0.283636
5	0.6	0.899333	0.212465	0.757576	0.331858
6	0.7	0.922000	0.252747	0.696970	0.370968
7	0.8	0.944333	0.324742	0.636364	0.430034
8	0.9	0.959667	0.408333	0.494949	0.447489

This highlights the tradeoff even more. Accuracy and precision comes at the cost of recall. The best balance comes at a threshold value of 0.6. Using this as the value for the optimized Logistic Regression model, we can tabulate the results for comparison.

```
recall_comparison = [0.949494, 0.767676, 0.757576]
f1_comparison = [0.134478, 0.608, 0.331858]
model_comparison = ['Baseline Logistic Regression', 'Baseline Decision Tree', 'Optimized Logistic Regression']
comparison_df = pd.DataFrame({'Model': model_comparison, 'Recall': recall_comparison, 'F1-Score': f1_comparison})
comparison_df
```

	Model	Recall	F1-Score
0	Baseline Logistic Regression	0.949494	0.134478
1	Baseline Decision Tree	0.767676	0.608000
2	Optimized Logistic Regression	0.757576	0.331858

In comparison to the baseline Logistic Regression model, recall is compromised for better balance. This however still shows the model's limit, such that even with optimization, it cannot achieve the balance while maintaining precision in comparison to the Baseline Decision Tree Classifier, thereby reinforcing the Decision Tree Classifier as a better choice for the business problem.

Optimizing the Decision Tree Classifier

Instead of running loops such as those above to find the optimal hyperparameter settings, a grid search is used

```
# Running a grid search
# Defining the grid scope
param_grid = {
    'max_depth': [2, 5, 10, 15, None],
    'min_samples_split': [2, 4, 6, 8, 10],
    'min_samples_leaf': [1, 2, 3, 4, 5]
}
#The baseline model can be used
d_tree = DecisionTreeClassifier(random_state=random_state)

#Creating the searching greid
grid_search = GridSearchCV(estimator=d_tree, param_grid=param_grid,
cv=5)
grid_search.fit(X_train_clean, y_train_clean)

# Best parameters
print("Best Parameters:", grid_search.best_params_)

Best Parameters: {'max_depth': None, 'min_samples_leaf': 1,
'min_samples_split': 4}
```

Applying these to optimize the model

```

clf = DecisionTreeClassifier(random_state=random_state,
criterion='entropy', max_depth=None, min_samples_leaf=1,
min_samples_split=4)

#Training the model
optimized_clf = clf.fit(X_train_clean, y_train_clean)

#Generating predictions
y_hat_optimized_clf = clf.predict(X_test_clean)

#printing the accuracy and precision
print('Accuracy:', accuracy_score(y_test_clean, y_hat_optimized_clf))
print('Precision:', precision_score(y_test_clean,
y_hat_optimized_clf))

Accuracy: 0.9676666666666667
Precision: 0.506578947368421

#Appending new metrics to the comparison table
recall_comparison.append(recall_score(y_test_clean,
y_hat_optimized_clf))
f1_comparison.append(f1_score(y_test_clean, y_hat_optimized_clf))
model_comparison.append('Optimized Decision Tree')
# Updating the DataFrame
comparison_df = pd.DataFrame({'Model': model_comparison, 'Recall':
recall_comparison, 'F1-Score': f1_comparison})
comparison_df

```

	Model	Recall	F1-Score
0	Baseline Logistic Regression	0.949494	0.134478
1	Baseline Decision Tree	0.767676	0.608000
2	Optimized Logistic Regression	0.757576	0.331858
3	Optimized Decision Tree	0.777778	0.613546

The Optimized Decision Tree Classifier stands out as the best model, having slightly improved in both recall and f1 score from its baseline. Though it doesn't stand out in recall performance, it manages to achieve about a 78% recall while also maintaining a good balance (0.61 F1-score)

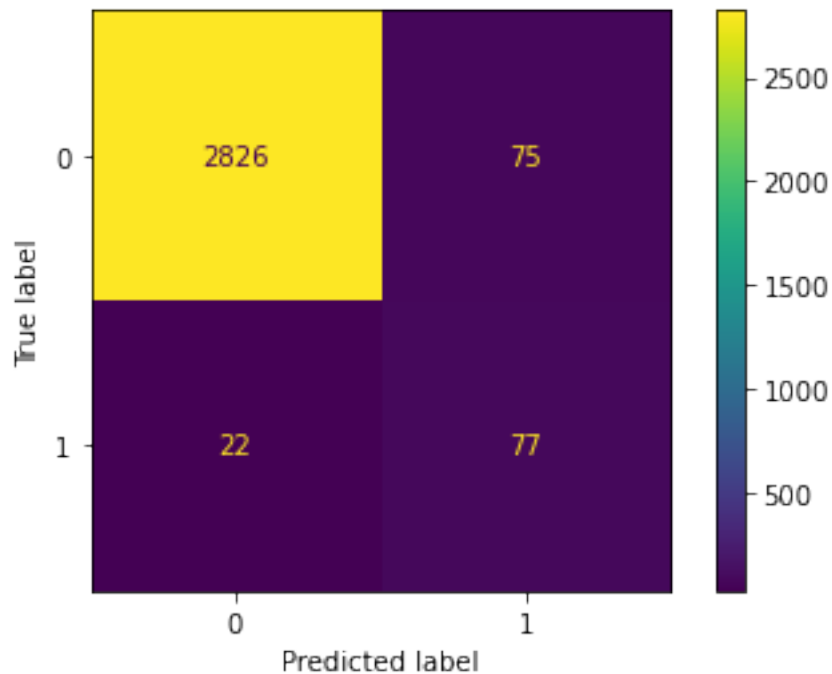
Taking a look at the confusion matrix once more for this optimization for better understanding;

```

cm = confusion_matrix(y_test_clean, y_hat_optimized_clf)

disp = ConfusionMatrixDisplay(cm).plot()
plt.show()

```

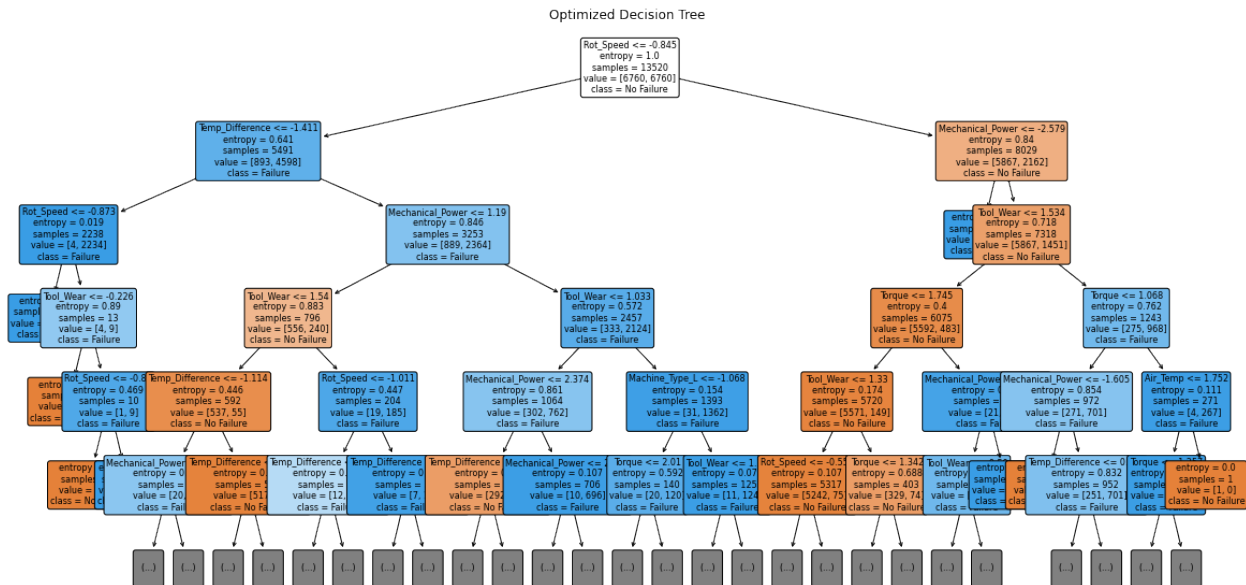


The only major improvement is that one more case of missed failure is accurately captured this time.

Taking a look at this final improved decision tree to see its logic behind the scenes

```
# Set figure size large enough to see just in case,
plt.figure(figsize=(20, 10))

plot_tree(
    optimized_clf,
    feature_names=X_train_encoded.columns,
    class_names=['No Failure', 'Failure'],
    filled=True, #to colour nodes by class
    rounded=True, #to round out shapes
    fontsize=8,
    max_depth=5 # to limit seeing all the nodes and leaves
)
plt.title("Optimized Decision Tree")
plt.show()
```



The comparison of baseline and optimized models highlights the trade-off between detecting failures (recall) and maintaining reliable predictions (F1-score). The baseline Logistic Regression achieved the highest recall (94.9%) but performed poorly overall due to extremely low precision, resulting in a weak F1-score. After optimization, its recall dropped to 75.8%, with a noticeable improvement in F1-score to 0.33, indicating a better balance but still limited practical reliability. The Decision Tree models demonstrate more stable performance, with the baseline Decision Tree achieved a strong F1-score (0.61) with moderate recall, and after optimization, it slightly improved both recall (77.8%) and F1-score (0.61), maintaining a balanced performance.

Overall, while Logistic Regression can be tuned to reduce its aggressiveness, it remains less effective at balancing recall and precision. The optimized Decision Tree provides the best combination of failure detection and actionable predictions, making it the most suitable model for this problem. ____

Model Interpretation

To ensure that the model is not only accurate but also meaningful in a practical engineering context, examining feature importance and model coefficients aids in identifying the key drivers of failure and assess whether these align with real-world mechanical principles, to answer the first businesses question. This section focuses on interpreting the trained models to understand how different features influence machine failure predictions.

```
#Using the feature_importances_ attribute from the Decision Tree Classifier
```

```
importances = optimized_clf.feature_importances_
```

```
#get the independent variable names (using the X_train encoded as the X_train_clean returns an error)
```

```
feature_names = X_train_encoded.columns
```

```
# Creating a pandas series to sort the values in descending order
```

```

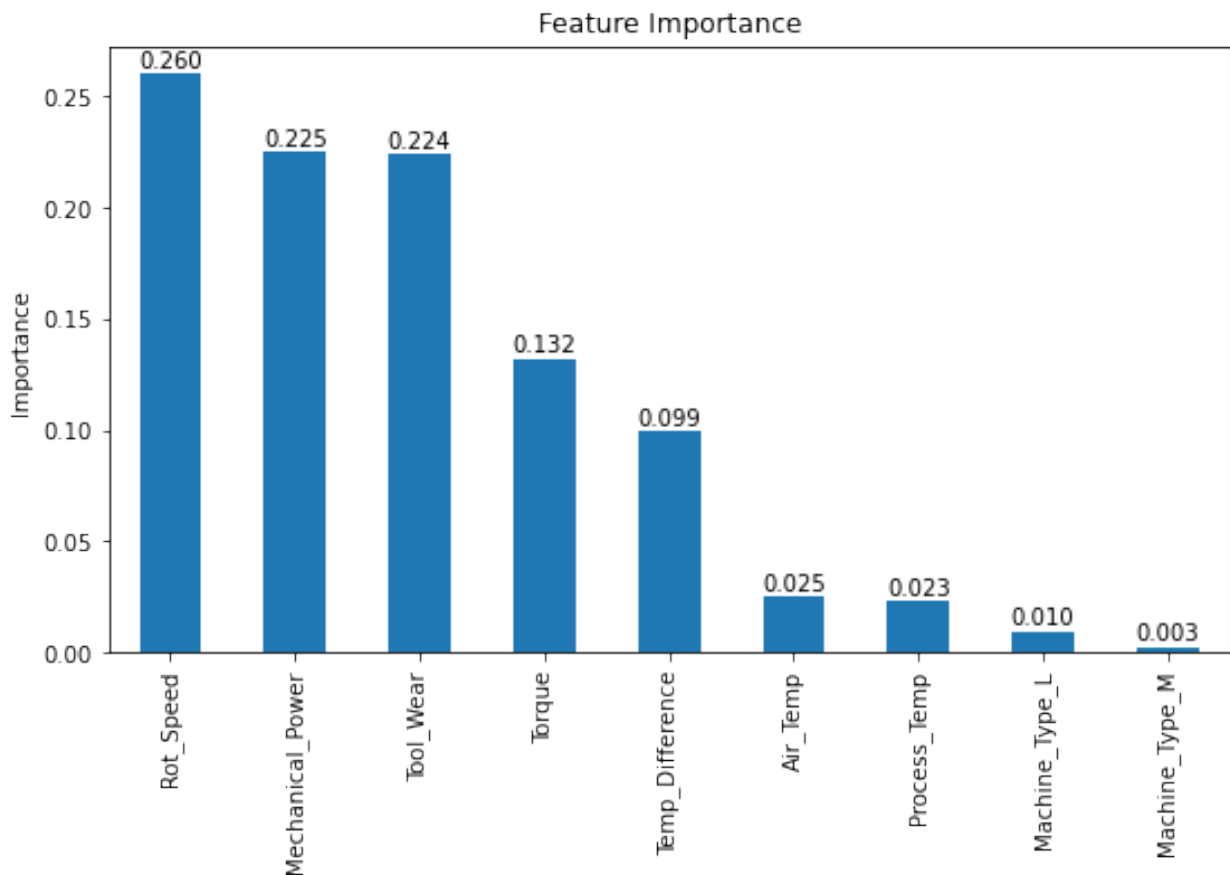
feat_imp = pd.Series(importances,
index=feature_names).sort_values(ascending=False)

#create and plot a bar graph using the pandas inbuilt plot function,
label the graph and y axis
ax = feat_imp.plot(kind='bar', figsize=(9,5))
plt.title('Feature Importance')
plt.ylabel('Importance')

# Add values on top of each bar
for i, v in enumerate(feat_imp):
    ax.text(i, v + 0.001, f"{v:.3f}", ha='center', va='bottom')

plt.show()

```

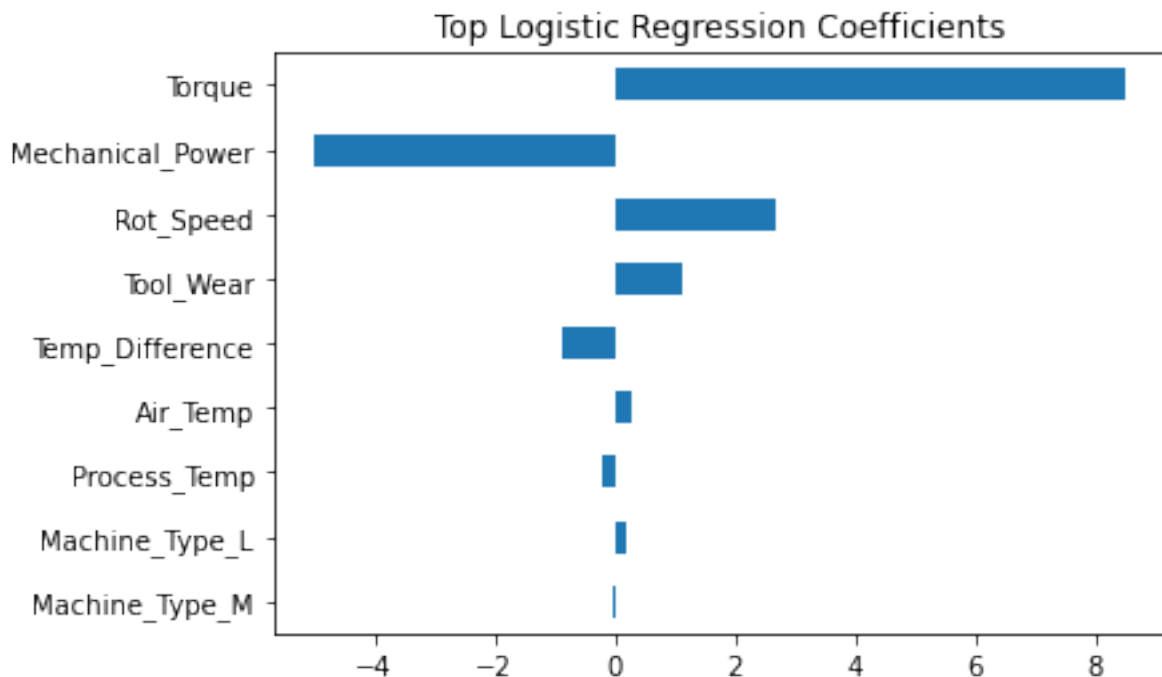


The Decision Tree identifies Rotational Speed, Mechanical Power and Tool Wear as the most important features, suggesting that mechanical stress and accumulated usage are the primary drivers of machine failure. Temperature-related variables also contribute, though to a lesser extent, indicating that thermal conditions play a supporting role. The type (size) of machine seems not to be a contributing factor, suggesting that the aforementioned factors contribute heavily across all machine types.

Taking a look at the coefficients for the logistic regression as well;


```
# Retrieving the coefficients into a pandas series similar to the
previous step
coefficients = pd.Series(improved_logreg.coef_[0],
index=X_train_encoded.columns).sort_values(key=abs, ascending=True)

#Plotting a graph as well
coefficients.head(10).plot(kind='barh')
plt.title('Top Logistic Regression Coefficients')
plt.show()
```



Positive coefficients for variables such as Torque, Rotational Speed and Tool Wear indicate that higher values increase the likelihood of machine failure. On the other hand, features with negative coefficients such as Mechanical power reduce failure probability. The magnitude of these coefficients reflects the strength of influence each variable has on the model's predictions. Similar to the decision tree, the Mechanical Power, Rotational Speed and Tool Wear heavily influence, however, Torque seems to be the most influential. Machine type and temperature are the least informative in predicting machine failure.

Insight

Across both models, Mechanical Power and Rotational Speed and Tool Wear consistently emerge as the strongest predictors of failure, indicating that machines operating under high mechanical load and prolonged usage are more prone to breakdown. This consistency strengthens confidence in these variables as key failure drivers.

Interpretation

From an engineering perspective,

- Mechanical Power (torque × rotational speed) captures the combined load and energy demand on the machine. Sustained high power output signals overloading, which accelerates component fatigue and wear.
- Rotational speed anomalies, either excessive or unstable, often indicate bearing degradation, misalignment, or process instability, all of which increase the likelihood of failure.
- Tool wear represents the cumulative degradation of the cutting edge over successive operations, directly reducing machining accuracy and raising the mechanical resistance the system must overcome.

Together, high power demand, irregular rotational speed, and advancing tool wear create a mutually reinforcing stress environment that aligns with known mechanical failure mechanisms i.e. excessive load, speed-related stress, and progressive tool deterioration push equipment toward breakdown. ____

Final Evaluation

As a final measure, to be able to determine whether the results are consistent and not dependent on the train test split, cross validation can be done to determine reliability and generalization ability, important for confirming that the model generalizes well to unseen data and can be trusted in real-world application. The Optimized Decision Tree Classifier, the best overall performing will be used as the final model.

```
# Using only the training data (before resampling was done) for
simplicity
cross_val_recall = cross_val_score(optimized_clf, X_train_scaled,
y_train, cv=5, scoring='recall')
cross_val_f1 = cross_val_score(optimized_clf, X_train_scaled, y_train,
cv=5, scoring='f1')
print("recall scores:", cross_val_recall)
print("f1 scores:", cross_val_f1)
print("\n")
print("Mean Recall score:", cross_val_recall.mean())
print("Mean F1 score:", cross_val_f1.mean())

recall scores: [0.79166667 0.625      0.75      0.72916667
0.85416667]
f1 scores: [0.8      0.70588235 0.74226804 0.76086957 0.84536082]

Mean Recall score: 0.7499999999999999
Mean F1 score: 0.7708761568275899
```

The final model was evaluated using 5-fold cross-validation to assess its stability across different subsets of the data. The cross-validation results show somewhat consistent performance across folds, indicating that the model is stable and not overly dependent on a specific train-test split. The average recall was 75% which is in the range of the target set in the business understanding section, showing an overall good ability to predict failures correctly. The

higher f1 score than previously seen can likely be attributed to the fact that the data was previously seen, since the model was trained on the same dataset it was cross validated against.

Overall, this, in combination with the results of the model on the test set (a recall of 77.8% and an F1-score of 0.61) convey that the model demonstrates a strong balance between detecting failures and maintaining prediction reliability, suggesting that the model generalizes well and is unlikely to suffer from significant overfitting in practical applications. ____

Conclusion

This project explored the use of machine learning to predict machine failure using sensor data, with the goal of reducing unplanned downtime and improving maintenance decision-making. Two models—Logistic Regression and Decision Tree—were developed and evaluated, with the Decision Tree ultimately providing the most balanced and practical performance.

Summary of Findings

The analysis showed that Rotational Speed, Machine Power(torque * rotational speed) and Tool Wear are the most significant indicators of machine failure, with temperature variables and machine types playing a secondary role. The Decision Tree model achieved a strong balance between recall (77.8%) and precision (0.61), allowing it to detect the majority of failures while limiting unnecessary maintenance actions. In contrast, Logistic Regression either over-predicted failures or required tuning that reduced its effectiveness in capturing them.

Addressing Business Questions

1. Which sensor conditions or combinations most strongly indicate an impending machine failure?

Rotational Speed, Machine Power and Tool Wear consistently emerged as the strongest predictors, indicating that mechanical load and accumulated usage are key drivers of failure.

2. Can a machine learning model distinguish between normal and failure-prone states with sufficient accuracy to be operationally trusted?

Yes. The optimized Decision Tree demonstrates that machine learning can reliably differentiate between normal and failure-prone conditions, achieving a strong balance between detecting failures and maintaining prediction reliability.

3. What is the right balance between catching all failures and controlling false alarms?

A balance that prioritizes high recall while maintaining moderate precision is most appropriate. Completely eliminating missed failures leads to excessive false alarms, while overly strict predictions risk missing critical failures. The Decision Tree achieves a practical middle ground suitable for real-world application.

Business Recommendations

The optimized Decision Tree model is recommended for deployment as a decision-support tool for predictive maintenance. It can be used to flag high-risk operating conditions and trigger inspections selectively, reducing unplanned downtime while avoiding excessive maintenance interventions. Operationally, integrating this model into monitoring systems would allow maintenance teams to act proactively rather than reactively.

Limitations and Assumptions

This analysis assumes that the dataset accurately represents real-world operating conditions. The use of synthetic balancing techniques such as SMOTE may not fully capture real failure patterns, and cross-validation results required careful interpretation to avoid leakage. Additionally, the model does not account for temporal dynamics, meaning it predicts failure risk based only on current conditions rather than trends over time.

Future Improvements

Future work could include incorporating time-series data to capture failure progression, exploring more advanced ensemble methods such as random forests, and integrating cost-sensitive learning to directly optimize for business impact. Further validation on real-world data would also strengthen confidence in deployment.

Final takeaway

The project demonstrates that machine learning can provide meaningful, interpretable, and actionable insights into machine failure, enabling a shift from reactive to proactive maintenance strategies.